EXPERIMENT 10

Performance Optimization in Zoho Creator

Name: Jenell S G | Register Number: 240701212

Subject: Low Code Application Development using Zoho Creator

Aim

To demonstrate performance optimization techniques in the EcoCycle e-waste application in Zoho Creator, particularly efficient record retrieval and workflow processing.

Introduction

Performance optimization in low-code applications means designing forms, workflows and scripts so that they use data and processing only as far as needed. Although the platform handles much of the underlying work, the way a workflow is written still decides how efficiently it runs.

Retrieving more records than required, or repeatedly processing records that are not relevant to the task, causes unnecessary data retrieval and processing. With the four e-waste entries of the sample data this is not noticeable, but as an institution records hundreds of items and collection requests it can slow the application. Efficient workflows therefore retrieve only the data they need, which supports scalable and maintainable applications.

Application Used

The practical uses the EcoCycle application, which records e-waste, collection requests and recycling activity. Deluge scripting is used in the form workflow of the E-Waste Entry form, which calculates the priority score and level and sends an e-mail for High priority items. The listings in this record are the reference implementations given in Appendix C of the project report; the weights of the priority factors are assumed. No upgrade-only feature is claimed as enabled or configured.

Performance Optimization Concept

The key optimization demonstrated is criteria-based record fetching. Instead of retrieving every record of a form, a criteria is given so that only the required record is returned. In the E-Waste Entry workflow this is written as:

```
rec = E_Waste_Entry[ID == input.ID];
```

- The workflow identifies the current record using **input.ID**, the ID of the e-waste entry that was just submitted.
- Only the matching E-Waste Entry record is retrieved and updated.
- Unrelated e-waste entries are not fetched or processed.
- This improves workflow efficiency and reduces unnecessary data processing.

Procedure

1. Open the EcoCycle application in Zoho Creator in Edit mode.
2. Go to the Workflow section and open Form workflows. The form E-Waste Entry is listed under Forms with workflows.
3. Open the workflow that calculates the priority. It is triggered on Created → Successful form submission (Figure 1).

4. When an entry is submitted, the workflow identifies the current record using input.ID.
5. The record is fetched with the criteria ID == input.ID, so only this record is retrieved.
6. The four factors (quantity, age, hazardous material and condition) give the priority score, which is compared with 8 and 4 to set High, Medium or Low.
7. The same record is updated with Priority Score, Priority Level, Collection Status = Pending and Recycling Status = Not Processed.
8. If the level is High, an e-mail notification is sent.

[Paste your own screenshot here]

Workflow page: form workflows of the E-Waste Entry form

Figure 1: Form Workflow of the EcoCycle Application (paste your screenshot of the Workflow section)

The overall flow of the workflow is shown in Figure 2 and the Deluge script is given in the next section.

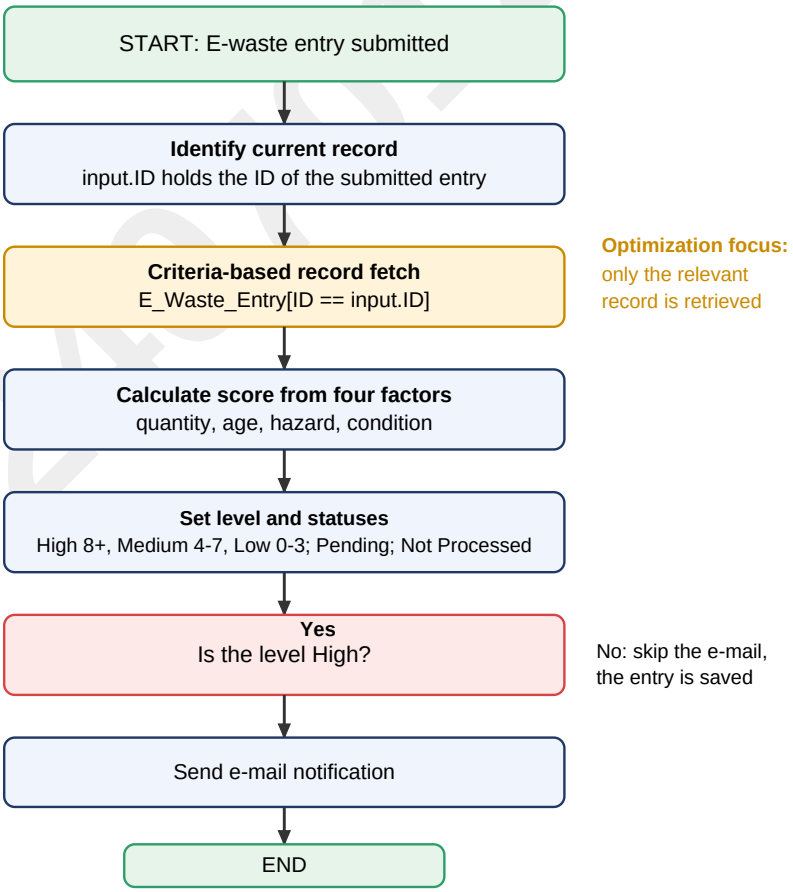

Figure 2: Optimized E-Waste Entry Workflow (Conceptual Diagram)

Deluge Code

The reference Deluge code of the workflow (Listing C.1 and C.3 of the report) is as follows:

```
rec = E_Waste_Entry[ID == input.ID];      // fetch only this record
score = 0;
if(input.Quantity >= 8)       { score = score + 3; }
else if(input.Quantity >= 4)  { score = score + 2; }
else if(input.Quantity >= 2)  { score = score + 1; }
if(input.Age_years >= 3)      { score = score + 2; }
else if(input.Age_years >= 1) { score = score + 1; }
if(input.Contains_Hazardous_Material == "Yes") { score = score + 3; }
if(input.Condition == "Not Working" || input.Condition == "Damaged")
{ score = score + 2; }
if(score >= 8)      { level = "High"; }
else if(score >= 4) { level = "Medium"; }
else                { level = "Low"; }
rec.Priority_Score    = score;
rec.Priority_Level    = level;
rec.Collection_Status = "Pending";
rec.Recycling_Status  = "Not Processed";
if(level == "High")
{
    sendmail
    [
        from    : zoho.adminuserid
        to      : zoho.adminuserid
        subject : "High-priority e-waste: " + input.E_Waste_Name
        message : "Quantity " + input.Quantity + ", priority score " + score + "."
    ]
}
```

The first statement fetches the record by its ID. The score is built from the four factors, the level is set, and the same record is updated. Only a High level triggers the e-mail. The factor weights are a reference scheme that respects the stated ranges; they are not copied from the application's own script.

Performance Analysis

The optimized approach is compared with a general inefficient approach below.

Approach	Steps
Before (inefficient approach)	Fetch all e-waste records → iterate through them → find the required record → process
Optimized (used in the workflow)	Fetch the required record using its ID → process directly

The inefficient approach retrieves and processes records that are not needed, whereas the optimized approach avoids this. Figure 3 is a conceptual diagram that shows how many records each approach retrieves. The “before” code is an illustrative example only and was not used in the application.

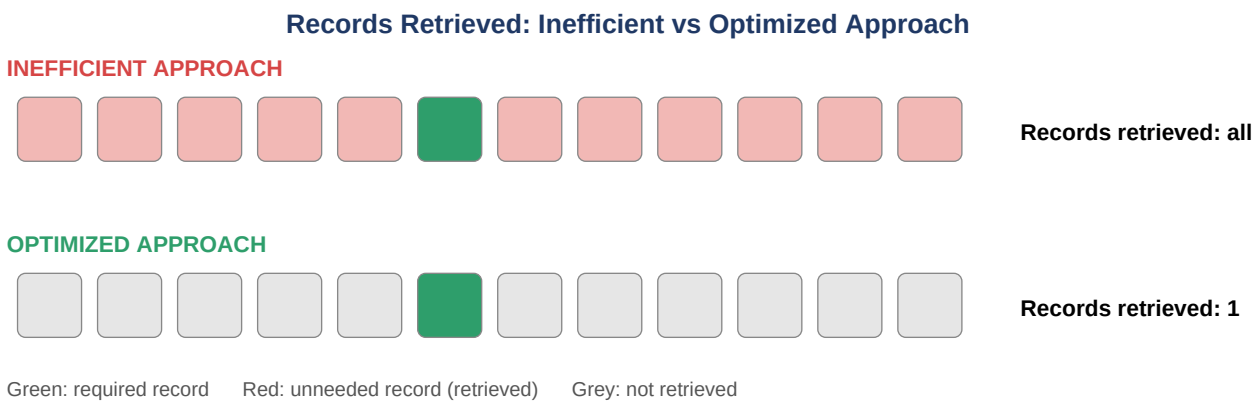

Figure 3: Records Retrieved by the Inefficient and Optimized Approaches (Conceptual Diagram, 12 sample records)

Illustrative comparison only (not used in the application):

```
wasteRecords = E_Waste_Entry[];
for each rec in wasteRecords
{
    if(rec.ID == input.ID)
    {
        // ...
    }
}
```

No benchmark was performed in this practical, so no measured improvement in execution time is claimed. The comparison is based on the amount of data retrieved and processed.

Optimization Techniques

General practices that contribute to performance in Zoho Creator applications are given in Figure 4, a conceptual diagram, and in the table that follows.

Figure 4: Performance Optimization Techniques (Conceptual Diagram)

Technique	Explanation
Criteria-based record fetching	Retrieve only the records required for the current operation. Used in this practical (E_Waste_Entry[ID == input.ID]).
Avoiding unnecessary record iteration	Do not fetch all records and loop through them when a criterion can be used.
Built-in functions and aggregate operations	Use built-in functions instead of repeated processing.
Indexed fields	Can improve retrieval of frequently queried data. Indexing may need platform configuration and was not enabled in EcoCycle.
Efficient workflows	Keep workflows simple and avoid nested or repeated operations.

Conclusion

Performance optimization matters in low-code applications because efficiency depends not only on the platform but also on how workflows are written. In EcoCycle, the E-Waste Entry workflow uses criteria-based retrieval with **E_Waste_Entry[ID == input.ID]**, so only the submitted record is fetched, scored and updated.

Targeted retrieval avoids fetching unrelated entries and looping through them, and simple workflow design reduces needless processing. These practices help EcoCycle cope with a growing number of e-waste entries and collection requests. No benchmark was performed, so the benefits are described as reduced data retrieval and processing and not as measured execution times.